Object Oriented Programming I

September 18, 2024

Content

Storage Classes

References and Reference Parameters

Default Arguments

Unary Scope Resolution Operator

Function Overloading

Multidimensional Arrays

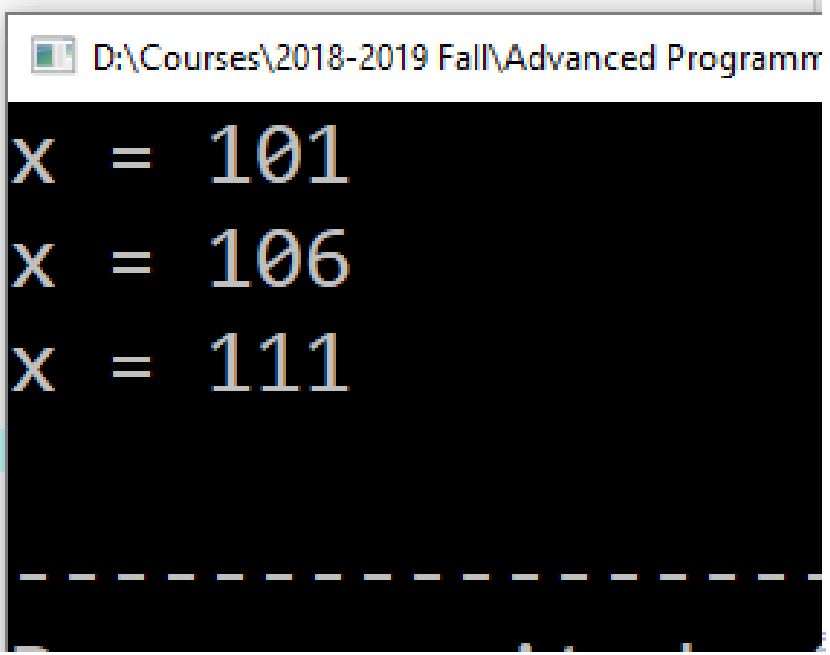
Static Storage Class

- Static storage class variables exist in memory from the point at which the program begins execution and last for the duration of the program.
- Such a variable is initialized once when its declaration is encountered.
- Variables declared **static** are still known only in the function in which they're declared, but, static variables retain their values when the function returns to its caller.
- The next time the function is called, the static variables contain the values they had when the function last completed execution.
- The following statement declares variable count to be static and to be initialized to 1

  static int count = 1;

Static Storage Class

```
11  #include <iostream>
12
13  using namespace std;
14
15  void useStaticVariable();
16
17  /* function main begins program execution
18  each program must include main function */
19  int main(void)
20  {
21      useStaticVariable();
22      useStaticVariable();
23      useStaticVariable();
24  }
25
26  void useStaticVariable()
27  {
28      static int x=101;
29
30      cout << "x = " << x << endl;
31
32      x=x+5;
33  }
```



Call-by-Value

- Two ways to pass arguments to functions in many programming languages are pass-by value and pass-by-reference.
- When an argument is passed by value, a copy of the argument's value is made and passed to the called function.
- Changes to the copy do not affect the original variable's value in the caller. This prevents the accidental side effects that so greatly hinder the development of correct and reliable software systems.

Reference Parameters

- With pass-by-reference,the caller gives the called function the ability to access the caller's data directly,and to modify that data.
- A reference parameter is an alias (nickname) for its corresponding argument in a function call.
- To indicate that a function parameter is passed by reference, simply follow the parameter's type in the function prototype by an ampersand (&)
- Use the same convention when listing the parameter's type in the function header.
- For example, the following declaration in a function header

  int &count

  when read from right to left is pronounced "count is a reference to an int."

Reference Parameters

- In the function call, simply mention the variable by name to pass it by reference.
- Then, mentioning the variable by its parameter name in the body of the called function actually refers to the original variable in the calling function, and the original variable can be modified directly by the called function.
- The following program compares pass-by-value and pass-by-reference with reference parameters.
- The "styles" of the arguments in the calls to function squareByValue and function squareByReference are identical.
- Both variables are simply mentioned by name in the function calls.
- Without checking the function prototypes or function definitions, it isn't possible to tell from the calls alone whether either function can modify its arguments.

Reference Parameters

```
10  #include <iostream>
11
12  using namespace std;
13
14  // Function Prototypes
15  int squareByValue(int x);
16  int squareByReference(int &y);
17
18  /* function main begins program execution each program must include main function */
19  int main(void)
20  {
21      int number1=2;
22      int number2=4;
23
24      // demonstrate squareByValue
25      cout << "number1 = " << number1 << " before squareByValue" << endl;
26      cout << "Value returned by squareByValue:" << squareByValue(number1) << endl;
27      cout << "number1 = " << number1 << " after squareByValue\n" << endl;
28
29      // demonstrate squareByReference
30      cout << "number2 = " << number2 << " before squareByReference" << endl;
31      squareByReference(number2);
32      cout << "number2 = " << number2 << " after squareByReference" << endl;
33
34      return 0; // indicates successful termination
35  }
36
37  int squareByValue(int x)
38  {
39      return x*x;
40  }
41
42  int squareByReference(int &y)
43  {
44      y=y*y;
45  }
```

References as Aliases with in a Function

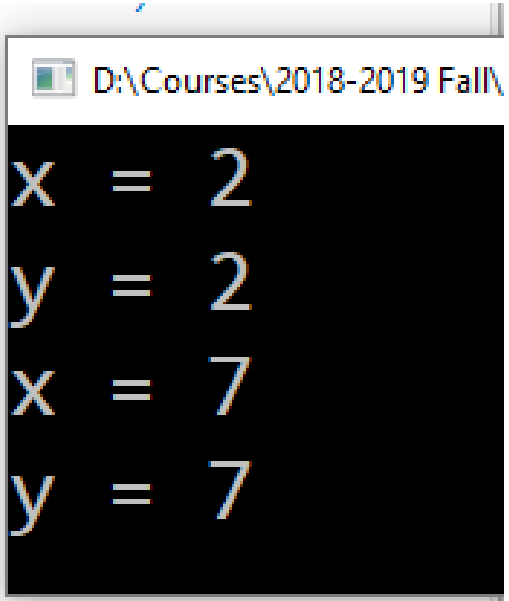
- References can also be used as aliases for other variables within a function although they typically are used with functions.
- For example, the following code

```
int count = 1;
int &cRef = count;
++cRef;
```

  increments variable count by using its alias cRef.
- **Reference variables must be initialized in their declarations and cannot be reassigned as aliases to other variables.**
- Once a reference is declared as an alias for another variable, all operations supposedly performed on the alias (i.e.,the reference) are actually performed on the original variable.
- **The alias is simply another name for the original variable.**

References as Aliases with in a Function

```
 7
 8  // includes input/output stream header
 9  // To output data to the screen
10  #include <iostream>
11
12  using namespace std;
13
14  /* function main begins program execution
15  each program must include main function */
16  int main(void)
17  {
18      int x=2;
19      int &y=x; // y refers to (is an alias for) x
20
21      cout << "x = " << x << endl << "y = " << y <<endl;
22
23      y= 7; // actually modifies x
24
25      cout << "x = " << x << endl << "y = " << y <<endl;
26
27      return 0; // indicates successful termination
28  }
```



Default Arguments

- It's common for a program to invoke a function repeatedly with the same argument value for a particular parameter.
- In such cases, you can specify that such a parameter has a default argument,i.e., a default value to be passed to that parameter.
- When a program omits an argument for a parameter with a default argument in a function call, the compiler rewrites the function call and inserts the default value of that argument.
- The following program demonstrates using default arguments to calculate a box's volume. The function prototype for boxVolume (line 7) specifies that all three parameters have been given default values of 1.

Default Arguments

```
10  #include <iostream>
11
12  using namespace std;
13
14  // function prototype that specifies default arguments
15  int boxVolume(int length = 1,int width= 1,int height = 1);
16
17  /* function main begins program execution
18  each program must include main function */
19  int main(void)
20  {
21      // no arguments use default values for all dimensions
22      cout << "The default box volume is: " << boxVolume();
23
24      // specify length; default width and height
25      cout << "\n\nThe volume of a box with length 10,\n"
26      << "width 1 and height 1 is: " << boxVolume(10);
27
28      // specify length and width; default height
29      cout << "\n\nThe volume of a box with length 10,\n"
30      << "width 5 and height 1 is: " << boxVolume(10,5);
31
32      // specify all arguments
33      cout << "\n\nThe volume of a box with length 10,\n"
34      << "width 5 and height 2 is: " << boxVolume(10,5,2) << endl;
35
36      return 0; // indicates successful termination
37  }
38
39  int boxVolume(int length,int width,int height)
40  {
41      return length*width*height;
42  }
```

Default Arguments

- Default arguments must be the rightmost (trailing) arguments in a function's parameter list.
- When calling a function with two or more default arguments, if an omitted argument is not the rightmost argument in the argument list,then all arguments to the right of that argument also must be omitted.
- Default arguments typically must be specified with the first occurrence of the function name, in other words in the function prototype.
- If the function prototype is omitted because the function definition also serves as the prototype,then the default arguments should be specified in the function header.

Unary Scope Resolution Operator

- It's possible to declare local and global variables of the same name.
- C++ provides the unary scope resolution operator (::) to access a global variable when a local variable oft he same name is in scope.
- The unary scope resolution operator cannot be used to access a local variable of the same name in an outer block.
- A global variable can be accessed directly without the unary scope resolution operator if the name of the global variable is not the same as that of a local variable in scope.

Unary Scope Resolution Operator

```
11  #include <iostream>
12
13  using namespace std;
14
15  int number = 7; // global variable named number
16
17  /* function main begins program execution
18  each program must include main function */
19  int main(void)
20  {
21      double number = 10.5; // local variable named number
22
23      // display values of local and global variables
24      cout << "Local double value of number= " << number
25      << "\nGlobal int value of number = " << ::number << endl;
26
27      return 0; // indicates successful termination
28  }
```

Unary Scope Resolution Operator

- You can also use the class scope operator to qualify class names or class data member names. If a class data member name is hidden, you can use it by qualifying it with its class name and the class scope operator.

```
11  #include <iostream>
12
13  using namespace std;
14
15  class X
16  {
17      public:
18          const static int count=10; // define static data member
19  };
20
21  int main ()
22  {
23      cout << X::count << endl;    // use static member of class X
24  }
```

Unary Scope Resolution Operator

- Note that the public data member count is specified by const static keywords.
- The data member is public so that it's accessible to the class's clients.
- Declaring count with the **const** qualifier indicates that this data member is constant which means that its value cannot be changed after being initialized.
- Keyword **static** in this variable declaration indicates that the data member is shared by all objects of the class.
- There are variables for which each object of a class does not have a separate copy. That is the case with **static** data members,which are also known as class variables.
- When objects of a class containing **static** data members are created, all the objects share one copy of the class's static data members.

Unary Scope Resolution Operator

- A static data member can be accessed within the class definition and the member-function definitions like any other data member.
- A public static data member can also be accessed outside of the class, even when no objects of the class exist,using the class name followed by the unary scope resolution operator (::) and the name of the data member.

Function Overloading

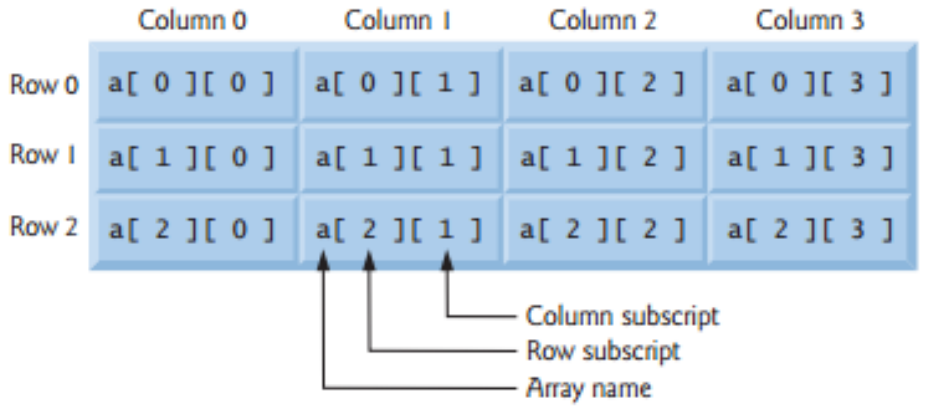
- C++ enables several functions of the same name to be defined, as long as they have different signatures. This is called function overloading.
- The C++ compiler selects the proper function to call by examining the number,types and order of the arguments in the call.
- Function overloading is used to create several functions of the same name that perform similar tasks, but on different data types.
- The following program uses overloaded square functions to calculate the square of an int and the square of a double.

Function Overloading

```
10  #include <iostream>
11
12  using namespace std;
13
14  // Function Prototypes
15  int square(int x);
16  double square(double y);
17
18  /* function main begins program execution each program
19  int main(void)
20  {
21      cout << square(7) << endl;
22      cout << square(7.5) << endl;
23
24      return 0; // indicates successful termination
25  }
26
27  int square(int x)
28  {
29      cout << "square of integer " << x << " is ";
30      return x*x;
31  }
32
33  double square(double y)
34  {
35      cout << "square of double " << y << " is ";
36      return y*y;
37  }
```

Two-dimensional Arrays

- Arrays with two dimensions often represent tables of values consisting of information arranged in rows and columns.
- To identify a particular table element, we must specify two subscripts.
- By convention,the first identifies the element's row and the second identifies the element's column. Arrays that require two subscripts to identify a particular element are called two-dimensional arrays or 2-D arrays.
- Every element in array a is identified in the figure by an element name of the form a[i][j],where a is the name of the array, and i and j are the subscripts that uniquely identify eachelement in a.



Two-dimensional Arrays

- A multidimensional array can be initialized in its declaration much like a one-dimensional array.
- For example, a two-dimensional array b with values 1 and 2 in its row0 elements and values 3 and 4 in its row 1 elements could be declared and initialized as follows:

  int b[2][2]= { {1,2} ,{3,4} };
- The values are grouped by row in braces.So, 1 and 2 initialize b[0][0] and b[0][1],respectively,and 3 and 4 initialize b[1][0] and b[1][1],respectively.
- If there are not enough initializers for a given row,the remaining elements of that row are initialized to 0.

  int b[2][2]= { {1} ,{3,4} };

initializes b[0][0] to 1, b[0][1] to 0, b[1][0] to 3 and b[1][1] to 4.

Two-dimensional Arrays

```
14  // Function Prototype
15  void printArray(const int a[][3]);
16
17  const int rows=2;
18  const int columns=3;
19
20  /* function main begins program execution each program must inc
21  int main(void)
22  {
23      int array1[ rows ][ columns ]= {{ 1, 2, 3 }, { 4, 5, 6 }};
24      int array2[ rows ][ columns ]= { 1, 2, 3, 4, 5 };
25      int array3[ rows ][ columns ]= {{ 1, 2 }, { 4 }};
26
27      cout << "Values in array1 by row are:" << endl;
28      printArray( array1 );
29
30      cout << "\nValues in array2 by row are:" << endl;
31      printArray( array2 );
32
33      cout << "\nValues in array3 by row are:" << endl;
34      printArray( array3 );
35
36      return 0; // indicates successful termination
37  }
38
39  void printArray(const int a[][3])
40  {
41      // loop through array's rows
42      for (int i=0;i<rows;++i){
43          // loop through columns of current row
44          for (int j=0;j<columns;++j)
45              cout << a[i][j] <<' ';
46
47          cout << endl;
48      }
49  }
50
```

- The first sublist initializes row 0 of the array to the values 1, 2 and 3; and the second sublist initializes row 1 of the array to the values 4,5 and 6.
- If the braces around each sublist are removed from the array1 initializer list, the compiler initializes the elements of row 0 followed by the elements of row 1, yielding the same result.
- The declaration of array2 provides only five initializers. The initializers are assigned to row 0, then row 1. Any elements that do not have an explicit initializer are initialized to zero, so array2[1][2] is initialized to zero.

Two-dimensional Arrays

- The program calls function printArray to output each array's elements.
- Notice that the function prototype and definition specify the parameter **const int** a[][columns].
- When a function receives a one-dimensional array as an argument,the array brackets are empty in the function's parameter list.
- The size of a two-dimensional array's first dimension is not required either, but all subsequent dimension sizes are required.
- The compiler uses these sizes to determine the locations in memory of elements in multidimensional arrays.
- All array elements are stored consecutively in memory,regardless of the number of dimensions. In a two-dimensional array, row 0 is stored in memory followed by row 1.
- Each row is a one-dimensional array. To locate an elementin a particular row,the function must know exactly how many elements are in each row so it can skip the proper number of memory locations when accessing the array.